School of Computer Science

COMP9120 Database Management Systems

Assignment 2: Database Application Development

Group assignment (13%)

Introduction

The objectives of this assignment are to gain practical experience in interacting with a relational database management system using an Application Programming Interface (API) (Python DB-API). This assignment additionally provides an opportunity to use more advanced features of a database such as functions.

This is a group assignment for teams of 3 members. It is assumed that you will continue in your Assignment 1 group. Any changes to the group composition after Thursday 23 April 2026 will not be acceptable.

Please also keep an eye on your email and any announcements that may be made on Ed.

Submission Details

The final submission of your database application is due at **11:59pm on Sunday 17th May (Week 11).** You should submit the items for submission (detailed below) via Canvas.

Items for submission

Please submit your solution to Assignment 2 in the 'Assignment' section of the unit's Canvas site by the deadline, including **EXACTLY SIX** files:

1. **An assignment coversheet** as a PDF document (.pdf file suffix) available for download on Canvas.
2. A file that **contains**: (A) the **GenAI prompts** you used and associated **genAI tools**, and (B) your **GenAI Comparison Report** that compares the generated and correct code for SQL and Python files (see below). Use a maximum of 1 page for each file (SQL and Python).
3. **The genAI SQL file (IOMCschema.sql)** containing the SQL statements necessary to generate the database schema and sample data. This must contain your base schema and insert SQL statements, **and your additional stored procedures (functions)** used by the application.
4. **The genAI Python file (database.py)** containing the Python code to access the database. As per this assignment's updated policy, the use of GenAI must be disclosed. You must ensure you understand and verify the code (see **Use of GenAI Tools**).
5. **Your (correct) SQL file** with the same requirement as **3.**
6. **Your (correct) Python file** with the same requirement as **4.**

Task 1: The IOMC Trials Management (ITM) System

In this assignment, you will be working with a modified version of your **IOMC** database from Assignment 1. This assignment builds upon the supplied IOMC database schema (see below). More specifically, your task is to implement an interface—referred to as the **ITM system**—through which an authorised **IOMC staff user** interacts to access and manipulate the IOMC database. Your main task is to handle requests for *reads* and *writes* to the database using your User Interface (UI). We first describe the main features that the ITM system must include from a UI perspective and then discuss where your database code needs to be implemented.

Logging In

The user is defined as any authorised **IOMC official**. The first form presented when starting the ITM system is the **Login** (Figure 1). This feature requires the user to enter their username and password to be validated prior to being successfully logged in to the system. Security features such as password encryption/hashing are out of scope for this assignment. The username should be **case-insensitive**. Once logged in, the user is directed to the **Athlete Trials Summary** page.

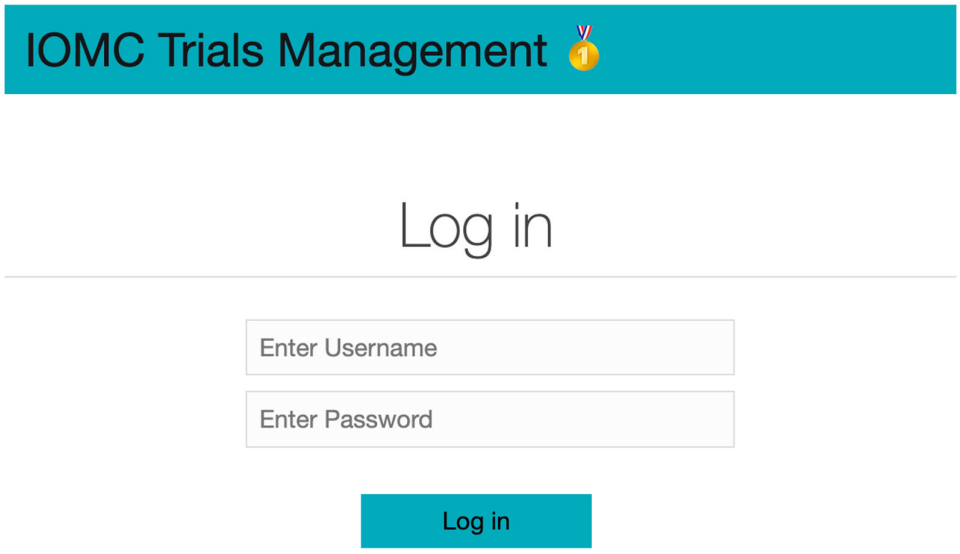

Figure 1. Login

Viewing Athlete Trials Summary

Once a user is logged in, they are directed to the **Athlete Trials Summary** page (Figure 2). The list must be ordered by **athlete last name** and **first name** in ascending order. The summary includes:

- **Athlete**: Full name, nationality, and athlete type (Professional/Amateur).

- **Trials Scheduled**: Number of upcoming trials for the athlete.
- **Trials Completed so far**: Number of completed trials.
- **Wins**: Count of trials the athlete **won** (each individual trial can be won by a single athlete).
- **Eligible Equipment**: Number of inspected items rated **good/very good/excellent** (only these are allowed for use).
- **Total Sponsorship ($)**: Sum of sponsorship payments received.
- **Last Trial Date**: Date when the athlete most recently **competed** (displayed in **Australian date convention** DD-MM-YYYY).

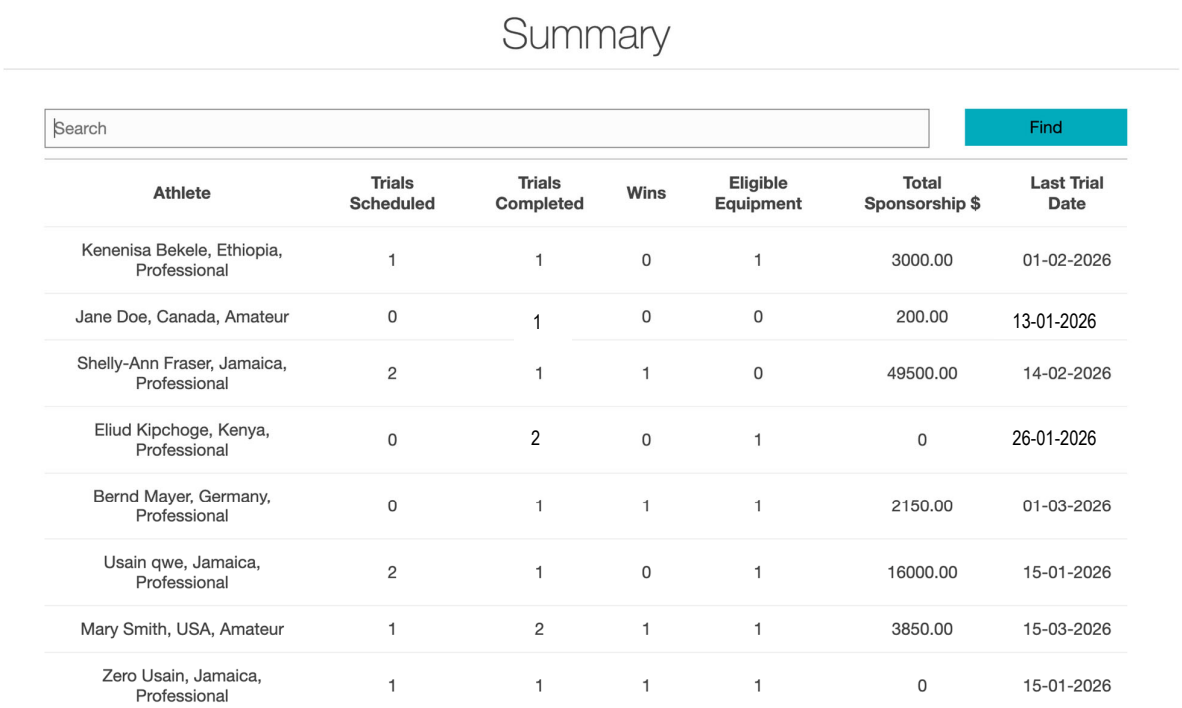

Figure 2. Viewing the Athlete Trials Summary Page

Finding Trials & Results

When a user selects a row in the **Athlete Trials Summary** table, they can view **detailed** records of each trial associated with that athlete (Figure 3 and Figure 4). Each trial record includes:

- **Trial ID**
- **Trial name**
- **Trial date**: When the trial happens, displayed in **Australian date convention** DD-MM-YYYY.
- **Athlete**: The full name of the participating athlete.
- **Is Winne**r: A status indicator that display “Yes” for the first place finisher or "No” for all other participants.
- **Official**: The full name of the IOMC official responsible for overseeing the trial participation.
- **Performance Note**: A quick record of observations the official noted down about the athlete; displays “N/A” if no notes were taken.

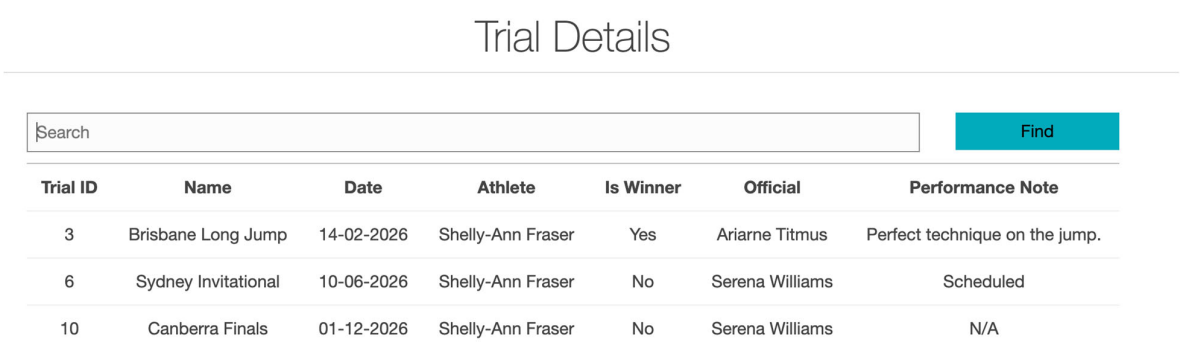

Figure 3. User clicks an Athlete row

Users can search the trial data by entering a keyword in the search field next to the **Find** button and clicking **Find**. The search retrieves and displays trial records containing the keyword in any of the following fields: **athlete full name**, **official's full name**, **trial name**, **trial date,** or **performance note**. The search is **case-insensitive**. Results must be ordered such that **upcoming trials** are listed at the top, followed by **completed** trials ordered by **trial date in ascending order**; completed trials with the same date are then sorted by **event name** in ascending order. **Requirements:**

- The search results must **exclude** any **completed** trial where the **trial date is older than 3 years** (from today's date).
- Trial date should be displayed in the **Australian date convention** (DD-MM-YYYY).

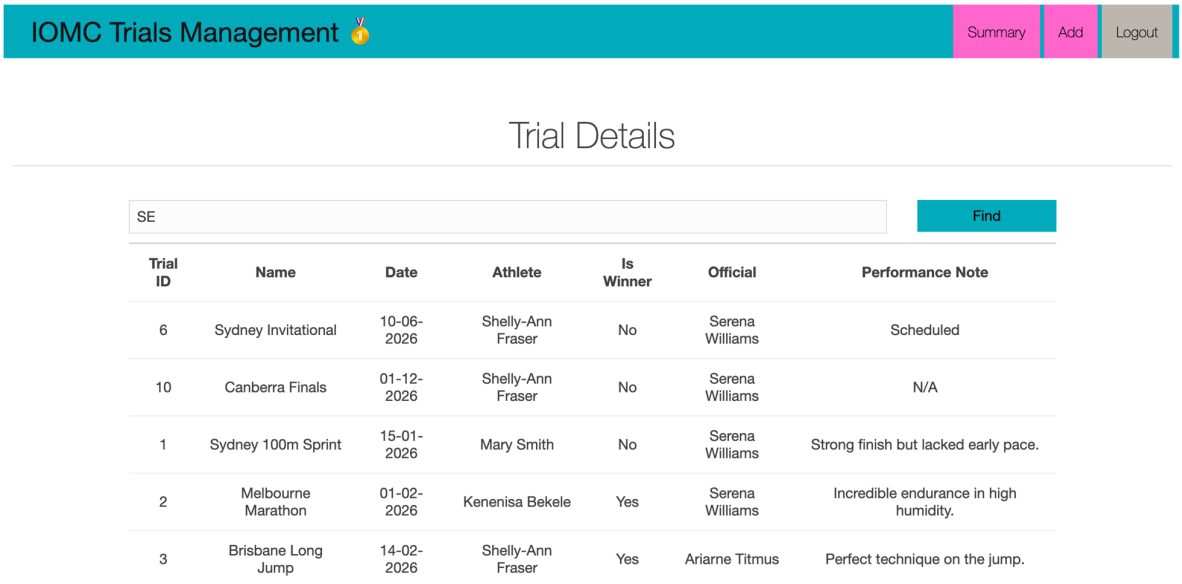

Figure 4. Finding Trials with search keyword "SE"

Adding a Trial Participation

Users may add a new **trial participation** by clicking on the **Add Trial** tab. In the **New Trial Participation** page, they must enter details and then click **Add Trial Participation** (Figure 5). A new participation is created with status **scheduled**

- **Athlete Username**: A valid athlete identifier from the database. (case insensitive)
- **Official Username**: A valid official identifier from the database. (case insensitive)
- **Trial Name:** The trial name. The system will verify if this specific trial event already exists for the selected date in the database; if it does not, a new trial event record would be created.
- **Date**: The scheduled date of the trial. **An athlete can only sign up for one event per day** (enforce this).

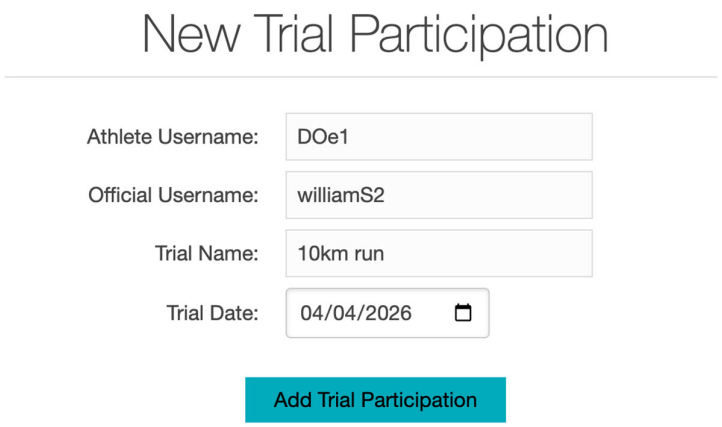

Figure 5. Adding a New Trial Participation

Updating a Trial Record

Users can update a trial record by modifying the data in the trial details screen (Figure 6), by clicking on **Update Trial**. A trial result cannot be recorded with a **future** date/time (from today's date).

- **Is Winner**: Mark **winner** (if applicable). **A trial can only be won by a single athlete**; enforce this.
- **Official Username**: A valid official identifier from the database. (case insensitive).

- **Performance Note**: A quick record of observations the official noted down about the athlete.

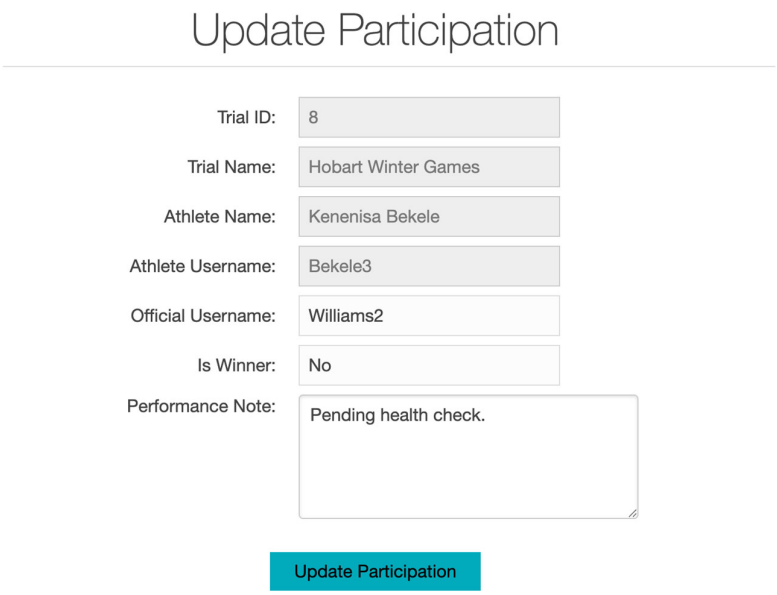

Figure 6. Updating a Trial Record

Database Interaction Code

The files that are needed for the Python version of the assignment are as follows:

1. **IOMCschema.sql**: Modify SQL statements from the assignment 1 solution to incorporate the new requirements.

   Link to the base sql schema.

   This file must contain the sql code needed to run to create and initialise the IOMC database before starting the application (base schema + sample data + stored procedures/functions).

2. **Skeleton project (Flask + psycopg2)**: a zip file encapsulating the Python project for the ITM system (provided on Canvas).

   Link to the skeleton project.

   Unzip to a folder (e.g., Assignment2_PythonSkeleton). The skeleton uses **Flask** for the GUI and **psycopg2** for PostgreSQL access. Install the modules similarly to the reference app:

   pip install psycopg2-binary

pip install flask

The main program starts in main.py. Provide the correct username/password and implement the missing DB access functions—**including the necessary SQL statements**—in the data layer database.py. The presentation layer uses HTML templates in templates/ and CSS in static/css. Routes are in routes.py. Run with python main.py and access via http://127.0.0.1:5001/ (or http://0.0.0.0:5002/). Stop the local web server to terminate.

Task 2: Functions Implementation

Core Functionality

Provide a complete implementation for **database.py**, and add/modify schema elements in **IOMCschema.sql** as needed. Implement these **five** functions (names can be used exactly as below):

1. **checkLogin(username, password)** (for login)
    - Case-insensitive username validation; return staff user details on success; reject otherwise.
2. **getAthleteTrialsSummary()** (for viewing the athlete trials summary)
    - Returns rows with athlete identity/type, trials scheduled/completed, wins, eligible equipment count, total sponsorship amount, and last trial date; order by last name, first name (ascending).
3. **findTrials(keyword, search_by_username=False)** (for finding trials)
    - if the search_by_username is False, perform case-insensitive search across athlete full name, official full name, event/trial name, trial date. Order with **upcoming first**, then **completed by date ascending**, then **event name ascending**. Exclude completed trials older than **3 years**. Show AU date format (DD-MM-YYYY).
    - If the search_by_username is True, perform case-insensitive search strictly for a specific athlete's username (triggered when a user selects an athlete from the summary table) . Order with trial_id,
4. **addTrialParticipation(athlete_username, trial_name, trial_date, official_username)** (for adding a trial participation)
    - Insert the participation; enforce **"one event per day per athlete"** ; enforce "one participation per trial". Reject invalid input.
5. **updateTrialResult(trial_id, athlete_username, is_winner, officials_update, perfroamnce_note)** (for updating a trial record)
    - Update performance/outcome; enforce **single winner per trial**. Reject invalid input.

Important: The corresponding actions must be **implemented by issuing SQL queries** to the DBMS (and/or invoking your stored procedures). Avoid implementing core query logic purely in Python when an SQL solution is intended. This mirrors the intent of the reference application assignment.

No additional Python modules or libraries should be imported.

In this assignment, the use of generative AI tools **is permitted**, including for **generating Python and SQL code**, provided the following requirements are met (reproduced in the style and placement of the original policy, but updated to allow code generation):

✔ **Permitted Uses**

You **may** use GenAI tools to: generate Python for database.py, write SQL statements, define schema/triggers/functions, provide helper snippets, and offer explanations of relevant concepts.

Requirements When Using GenAI

1. You must **review, test, and understand** all GenAI-generated code.
2. You are **fully responsible** for its correctness.
3. You must include a **GenAI Disclosure** (tools used, purposes).
4. You must include your **GenAI prompts** as indicated above.

GenAI Disclosure (Required)

Provide: the tools used, purposes (e.g., "generated SQL for summary query"), and a declaration that you **reviewed, modified, and verified** all GenAI code. Lack of disclosure yields **0** for the GenAI rubric component

Additional GenAI Comparison Questions (Required)

Submit a **1-page (max) analytic report** (appendix to the coversheet PDF) titled **GenAI_Comparison_Report** answering:

1. **Issues in GenAI output** — Identify incorrect/missing/suboptimal code or SQL (e.g., wrong JOINs, missing constraints, violating single-winner rule, poor error handling). Cite short snippets as needed.
2. **Why they are incorrect** — Justify using the IOMC domain rules and assignment requirements (e.g., participation, official minimum, 1-per-day scheduling).
3. **How you corrected them** — Explain your fixes and why they satisfy the specification.
4. **Usefulness vs limitations** — Reflect on where GenAI helped and where it struggled in this assignment's context.

Marking

This assignment is worth **13%** of your final grade for the unit of study. Your group's submission will be marked according to the attached rubric.

Group member participation

If members of your group do not contribute sufficiently, you should alert the unit coordinator as soon as possible. The course instructor has the discretion to scale the group's mark for each member as follows:

Percentage of contribution	Proportion of final grade received
< 5% contribution	0%
5 - 10% contribution	20%
11 - 15% contribution	40%
16 - 20% contribution	50%
21 - 24% contribution	60%
25 - 28% contribution	80%
29 - 30% contribution	90%
> 30% contribution	100%

Note: The above table assumes that each group will have 3 members, so, on average, around 33% contribution is expected from each member of the group. In special case, if a group has less than 3 members then the contribution percentage will be adjusted accordingly. You must justify your contribution percentage by providing a detailed explanation of your individual contribution on the assignment coversheet mentioned before. You must also regularly record and maintain a diary of your group meetings and discussions on Canvas. Furthermore, we may run random face-to-face interviews to understand and justify your contribution, if needed.

Marking Rubric

Your submissions will be marked according to the following rubric, with a maximum possible score of **13** points.

	0 – 1 marks	**1 – 1.5 marks**	**2 marks**
1**. Login Functionality**	Incorrect login handling; case-insensitivity not implemented; major errors.	Correct handling for a test user; minor issues with validation or error reporting.	All valid users authenticate correctly; invalid logins rejected; case-insensitive username fully implemented.
2. Athlete Trials Summary	Missing required fields or incorrect logic; ordering incorrect.	Summary generally correct with some minor inconsistencies (e.g., aggregates or formatting).	Fully correct: all fields, AU date format, correct aggregates (wins, eligible equipment, sponsorship totals), correct ordering.
3. Find Trials Functionality	Search fails or ignores requirements; incorrect ordering; missing filters.	Search works for common inputs; partially correct ordering and filters.	Fully correct: case-insensitive search, correct upcoming→completed ordering, hides trials older than 3 years, AU date formatting.
4. Add Trial Participation	Fails to add participation or violates key rules.	Adds valid records but may mishandle some constraints.	Fully correct: enforces "one event per day," validates athlete & officials, handles all edge cases.

5. Update Trial Record	Updates incomplete or incorrect; required constraints missing.	Updates most fields correctly; minor issues with winner or official validation.	Fully correct: enforces single winner, prevents future dates, maintains ≥1 official, correct update logic.

Supporting Components (3 marks total)

	0 marks	**1 mark**
6. Stored Procedures / Functions	Missing required stored procedures OR not used by Python functions OR implemented incorrectly.	At least two stored procedures/functions implemented correctly AND invoked from two of the five major functions.
7. Record Keeping of Group Discussions	One or more issues reported or found with group member contribution, or with maintaining records of group meetings and discussions regularly on Canvas.	No issue reported or found with group member contribution. All group members participate and regularly maintain a diary of group meetings and discussions on Canvas.

	0 marks	**0.5 mark**
8. GenAI Usage	Fails to disclose GenAI use, discloses dishonestly, or evidently does not understand GenAI-generated code.	Complete, honest disclosure; includes prompts; demonstrates understanding and correct verification of all GenAI-generated code.
9. GenAI Comparison Report	No report OR identifies few/no issues OR fails to justify corrections.	Clear, succinct, and comprehensive analysis (max 1 page): identifies key issues, explains why they were incorrect, and describes how they were fixed.